FE 520 Introduction to Python for Financial Applications
Assignment 1 100 Points

Question 1 20 Points

Use this data set to solve the questions below:

```
raw = " aapl ,msft, GOOG ;BRK.B| tsla  ,  spy  ,aapl,  Ibm ; nvda, brk-b,   ^GSPC "
universe = ["AAPL","MSFT","GOOGL","GOOG","BRK.B","TSLA","SPY","IBM","NVDA","^GSPC","META","AMZN"]
```

1. Produce tickers: uppercase tickers from raw, where all [,;|] are treated as commas, whitespace is trimmed, blanks removed, and BRK-B is canonicalized to BRK.B (case-insensitive). No loops. re allowed. (4 Points)
   Sample Output-
   ```
   ['AAPL','MSFT','GOOG','BRK.B','TSLA','SPY','AAPL','IBM','NVDA','BRK.B','^GSPC']
   ```

2. Produce unique ordered: first occurrence of each symbol from tickers, order preserved. No loops. (4 Points)
   Sample Output-
   ```
   ['AAPL','MSFT','GOOG','BRK.B','TSLA','SPY','IBM','NVDA','^GSPC']
   ```

3. Produce valid_universe: keep only symbols that exist in universe, order as in unique_ordered. (4 Points)
   Sample Output-
   ```
   ['AAPL','MSFT','GOOG','BRK.B','TSLA','SPY','IBM','NVDA','^GSPC']
   ```

4. Produce dropped: the set of symbols in unique_ordered not found in universe as a sorted list (may be empty). (4 Points)
   Sample Output-
   ```
   []
   ```

5. Produce a single string status: (4 Points)
   Sample Output-
   ```
   "VALID:^GSPC,AAPL,BRK.B,GOOG,IBM,MSFT,NVDA,SPY,TSLA; DROPPED:"
   ```

Question 2 — 20 Points

Use this data set to solve the questions below:

```python
targets = """
AAPL: 25%, MSFT:25 % , nvda :  30% ,  GOOG:10%,  brk-b:10%
"""
notes = [
 "AAPL split-adjusted; BUYbacks ongoing",
 "MSFT AI capex: strong; HOLD raised to BUY",
 "NVDA valuation-debated; DOWNGRADE risk",
 "GOOG ads cyclical-HOLD",
 "BRK.B cash pile record; BUY"
]
```

1. Produce pairs: list of (ticker, weight_float) sorted alphabetically by ticker. Canonicalize BRK-B → BRK.B. No loops. (4 Points)
   Sample Output:

```
[('AAPL',25.0),('BRK.B',10.0),('GOOG',10.0),('MSFT',25.0),('NVDA',30.0)]
```

2. Produce weights_status:
   If total ≈ 100.00 within ±0.01 → "WEIGHTS OK (100.00)" (two decimals)
   else → "WEIGHTS ERROR (sum=xx.xx)" (4 Points)
   Sample Output:

```
"WEIGHTS OK (100.00)"
```

3. From notes, produce three sorted lists: buy_list, hold_list, risk_flags. Match case-insensitive words: BUY (or BUYBACKS), HOLD, DOWNGRADE. No explicit loops. (4 Points)
   Sample Output:

```
buy_list   -> ['AAPL','BRK.B','MSFT']
hold_list  -> ['GOOG','MSFT']
risk_flags -> ['NVDA']
```

4. Produce a 3-line memo (you may use textwrap.fill to wrap to width 60 if desired, but keep one line per item): (4 Points)
   Sample Output:

```
Coverage: AAPL,BRK.B,GOOG,MSFT,NVDA
BUY: AAPL,BRK.B,MSFT
HOLD: GOOG,MSFT | RISK: NVDA
```

5. Produce CSV header csv_header and one alphabetical row csv_row from the following data: (4 Points)

```
"Ticker","Target(%)","BUY","HOLD","RISK"
"AAPL","25.00","Y","",""
```

Sample Output:

```
"Ticker","Target(%)","BUY","HOLD","RISK"
"AAPL","25.00","Y","",""
"BRK.B","10.00","Y","",""
"GOOG","10.00","","Y",""
"MSFT","25.00","Y","Y",""
"NVDA","30.00","","","Y"
```

Question 3 **20 Points**

Use the data to solve questions below:

```
sec = """
We returned $29.5 billion to shareholders via buybacks and dividends in FY2024.
Operating margin expanded by 120 bps to 31.2%. Cash and marketable securities totaled $61.8 billion.
We reduced diluted shares outstanding by ~1.8%. Long-term debt declined to $105.3 billion.
"""
```

1. Produce dollar_billions: unique numeric strings in billions from $29.5 billion, $61.8 billion, $105.3 billion (e.g., ['105.3','29.5','61.8']) sorted lexicographically. One re.findall + set + sort, no loops. (4 Points)
   Sample Output:

```
['105.3','29.5','61.8']
```

2. Produce percents: keep percent terms like 31.2%, 1.8% in original order and deduplicate (first occurrence kept). No explicit loops. (4 Points)
   Sample Output:

```
['31.2%','1.8%']
```

3. Produce metrics_line: (4 Points)
   Sample Output:

```
"BILLIONS:[105.3,29.5,61.8] | PCTS:[31.2%,1.8%]"
```

4. Produce sec_approx: replace any ~ that immediately precedes a percent with ≈ (e.g., ~1.8% → ≈1.8%). (4 Points)
   Sample Output:

```
"... reduced diluted shares outstanding by ≈1.8% ..."
```

5. Produce sec_clean: collapse whitespace, ensure normal spacing, and end with a single newline. Sentence casing is optional here; whitespace normalization is required. (4 Points)
   Sample Output:

We returned $29.5 billion to shareholders via buybacks and dividends in FY2024. Operating margin expanded by 120 bps to 31.2%. Cash and marketable securities totaled $61.8 billion. We reduced diluted shares outstanding by ≈1.8%. Long-term debt declined to $105.3 billion.

Question 4. **20 Points**

Use this data set to solve the questions below:

```
csv_text = """Date,Close
2025-09-12,243.20
2025-09-15,246.84
2025-09-16,245.10
2025-09-17,247.42
2025-09-18,249.90
"""
ticker = "AAPL"
```

1. Produce rows (data lines only) and closes as list[float]. No loops. (4 Points)
   Sample Output:

```
rows   -> ['2025-09-12,243.20','2025-09-15,246.84','2025-09-16,245.10','2025-09-17,247.42','2025-09-18,249.90']
closes -> [243.2, 246.84, 245.1, 247.42, 249.9]
```

2. Produce rets with simple returns (p1/p0 - 1) using slicing + zip. No loops. (4 Points)
   Sample Output:

```
[0.014967105263157 92, -0.007049100631988381, 0.009465524275805715, 0.010023441920620879]
```

3. Produce mean_ret, min_ret, max_ret and their strings mean_str, min_str, max_str formatted as signed percentages with two decimals. No loops. (4 Points)
   Sample Output:

```
mean_str -> +0.69%
min_str  -> -0.70%
max_str  -> +1.50%
```

4. Produce perf_line exactly: (4 Points)
   Sample Output:

```
"AAPL,4,+0.69%,-0.70%,+1.50%"
```

5. Produce a 3-line text block report_text: (4 Points)
   Sample Output:

```
AAPL — Daily Close Snapshot
Dates: 2025-09-12 → 2025-09-18
Observations: 5 (returns: 4) | Mean/Min/Max: +0.69% / -0.70% / +1.50%
```

Question 5 — 20 Points

Use this data set to solve the questions below:

```
headlines = [
 "AAPL gains as Wedbush boosts price-target; BUY reiterated",
 "msft slips after DOJ update; Neutral → HOLD; PT trimmed",
 "GOOG jumps on cloud deal wins—Upgrade to BUY!",
 "NVDA downgraded to HOLD; valuation 'stretched'? ",
 "brk.b: Berkshire raises cash pile—Sell off in cyclicals?",
 "Spy edges higher; flows strong; BUY-the-dip chatter",
]
signal_buy  = {"BUY","UPGRADE","ACCUMULATE","OVERWEIGHT"}
signal_sell = {"SELL","DOWNGRADE","UNDERWEIGHT","REDUCE"}
signal_hold = {"HOLD","NEUTRAL","MARKET PERFORM"}
```

1. Produce clean_lines: each line uppercased, Unicode dashes replaced with -, and exactly one space after , ; : - when followed by a letter. No loops. re allowed. (4 Points)
   Sample Output:

```
['AAPL GAINS AS WEDBUSH BOOSTS PRICE-TARGET; BUY REITERATED',
 'MSFT SLIPS AFTER DOJ UPDATE; NEUTRAL → HOLD; PT TRIMMED', ...]
```

2. Produce a flat token list tokens using one regex per line that: (4 Points)
   - keeps BRK.B and ^GSPC intact,
   - removes standalone punctuation,
   - returns only alphanumeric/period/caret tokens. (No loops.)
     Sample Output:

['AAPL','GAINS','AS','WEDBUSH','BOOSTS','PRICE-TARGET','BUY','REITERATED','MSFT','SLIPS',...]

3. Count case-insensitive hits for BUY/SELL/HOLD words as buy_hits, sell_hits, hold_hits. Use collections.Counter allowed here. (4 Points)
   Sample Output:

```
buy_hits=3, sell_hits=1, hold_hits=2
```

4. Produce mentioned: sorted list of tickers from tokens that are present in universe (use exact match of tokens). (4 Points)
   Sample Output:

```
['AAPL','BRK.B','GOOG','MSFT','NVDA','SPY']
```

5. Produce headline_summary exactly: (4 Points)
   Sample Output:

"6 headlines | BUY: 3, HOLD: 2, SELL: 1 | Tickers: AAPL,BRK.B,GOOG,MSFT,NVDA,SPY"